1. Czym jest testowanie oprogramowania? W jakim celu testujemy?

Testowanie oprogramowania to proces sprawdzania, czy aplikacja działa zgodnie z wymaganiami, zachowuje się prawidlowo w różnych sytuacjach i nie zawiera błędów, które mogłyby utrudnić jej użytkowanie.

Testowanie nie polega wyłącznie na szukaniu błędów. Pomaga również ocenić jakość aplikacji, bezpieczeństwo i jej gotowość do wdrożenia.

Główne cele testowania:

- Wykrywanie błędów
- Weryfikacja wymagań
- Zapewnienie jakości
- Ograniczanie ryzyka
- Sprawdzanie bezpieczeństwa
- Zapobieganie regresji

2. Jakie są rodzaje testów?

Testy można podzielić na kilka grup w zależności od tego, co sprawdzają i w jaki sposób są wykonywane.

A. Podział ze względu na poziom testowania

1. Testy jednostkowe
    - Sprawdzają małe fragmenty kodu, np. pojedynczą funkcję, metodę lub klasę.
2. Testy integracyjne
    - Sprawdzają, czy różne moduły współpracują ze sobą poprawnie.
3. Testy systemowe
    - Sprawdzają całą aplikację jako kompletny system.
4. Testy akceptacyjne
    - Sprawdzają, czy aplikacja spełnia wymagania biznesowe i potrzeby użytkownika.

B. Podział ze względu na cel testowania

- Testy funkcjonalne – sprawdzają, co aplikacja robi, np. rejestrację, wyszukiwanie i logowanie.
- Testy eksploracyjne – tester samodzielnie poznaje aplikację, eksperymentuje i szuka nieoczekiwanych zachowań.
- Testy wydajnościowe – sprawdzają szybkość działania i zachowanie aplikacji pod obciążeniem.
- Testy bezpieczeństwa – szukają luk, np. możliwości obejścia autoryzacji.
- Testy użyteczności – sprawdzają, czy aplikacja jest intuicyjna i wygodna w obsłudze.
- Testy kompatybilności – sprawdzają działanie na różnych przeglądarkach, systemach i urządzeniach.

C. Testy manualne i automatyczne

Testy manualne – nie jest to robione automatycznie, robią to ludzie którzy testują i sprawdzają co może nie działać

Testy automatyczne – tutaj skrypty porównują wyniki z planowanymi oczekiwaniami.

3. Testowanie aplikacji webowych i mobilnych

W aplikacjach webowych najczęściej testuje się formularze logowanie, działanie strony, API i zgodność z różnymi przeglądarkami. W aplikacjach mobilnych sprawdzamy też działanie na różnych telefonach, powiadomienia i opcje braku internetu.

Popularne narzędzia:

- Playwright – automatyzuje testowanie stron internetowych.
- Selenium – pozwala automatycznie obsługiwać przeglądarki i testować aplikacje webowe.
- Postman – służy do sprawdzania API, czyli komunikacji z serwerem.
- Jest – służy do testów jednostkowych w JavaScript.
- Appium – pozwala automatyzować testy aplikacji mobilnych na Androidzie i iOS.
- Espresso – służy do testowania interfejsu aplikacji na Androidzie.
- XCTest / XCUITest – narzędzia do testowania aplikacji na iOS.

4. Testy jednostkowe – co testujemy i czy warto testować wszystko?

Testy jednostkowe sprawdzają małe fragmenty kodu, np. pojedyncze funkcje.

W aplikacjach webowych testujemy np. walidację formularzy, logowanie, obliczanie cen. W aplikacjach mobilnych dodatkowo logikę ekranów, zarządzanie stanem i przetwarzanie danych.

Nie warto testować wszystkiego na siłę, tak naprawdę zależy to od wielkości projektu i jego ważności bo jeśli jest to na przykład strona jakiegoś ważnego banku gdzie przelewy i te sprawy są na wagę złota jest dosyć mocno zwrócona uwaga na testowanie i możliwości włamania się, by odpowiednio dobrać zabezpieczenia. Najważniejsze są funkcje, w których błąd może spowodować poważny problem, np. płatności, logowanie czy zapisywanie danych. Testy powinny dawać pewność, że aplikacja działa poprawnie.

5. Co powinien robić dobry test?

Dobry test powinien:

- Sprawdzać konkretną funkcjonalność
- Być czytelny i łatwy do zrozumienia
- Dawać powtarzalny wynik
- Być niezależny od innych testów
- Sprawdzać, czy wynik jest zgodny z oczekiwaniami
- Wykrywać błędy, a nie tylko przechodzić bez problemów

Przykładowo, jeśli testujemy dodawanie dwóch liczb, to dla 2 + 3 powinniśmy sprawdzić, czy wynik wynosi 5.

6. Czym są edge case?

Edge case to nietypowa lub skrajna sytuacja, w której aplikacja może zachować się nieprawidłowo.

Przykłady:

- Użytkownik wysyła pusty formularz.

- Wpisuje bardzo długi tekst.
- Podaje kwotę równą zero.
- Traci internet podczas zapisywania danych.
- Klika przycisk kilka razy pod rząd.

Testowanie takich sytuacji pomaga znaleźć błędy, których nie widać podczas normalnego korzystania z aplikacji

7. Na czym polega mockowanie?

Mockowanie polega na zastępowaniu prawdziwych zależności specjalnymi zamiennikami, które zachowują się tak, jak ustalimy w teście.

Przykład: testujemy aplikację pogodową. Zamiast pobierać prawdziwą temperaturę z internetu, ustawiamy mock, który zawsze zwraca 20°C. Dzięki temu możemy sprawdzić działanie aplikacji bez połączenia z serwerem.

Mockowanie przydaje się, gdy chcemy:

- Uniezależnić test od internetu i zewnętrznych usług.
- Symulować błędy serwera.
- Sprawdzić, czy dana funkcja została wywołana.
- Uniknąć wykonywania prawdziwych płatności lub wysyłania e-maili.